# StageSight: A River Camera That Checks Its Gauge

OpenCV 5 reads flood stage from USGS staff plates; a Claude agent on AWS decides which instrument to trust

Muhammad Waqar Saleem
Department of Civil and Environmental Engineering, University of Missouri, Columbia, MO, USA
msx9t@missouri.edu ORCID 0009-0004-4976-6979
OpenCV AI Competition 2026, powered by AWS

***Abstract*—A river gauge reports stage from a single pressure sensor; when that sensor drifts, freezes or goes silent, often during the flood it exists for, the record gives no sign. Many USGS gauges already carry a camera aimed at a staff plate, but its images are inspected by people, not measured. StageSight turns that camera into an independent second instrument. Classical OpenCV 5 operations and the OpenCV DNN text recognizer read the stage directly from the plate's printed marks and numbers, *without any gauge data*. On the 29 July 2026 flood at Hancock, NY (USGS 01421500; NWS minor flood, 18.08 ft peak), the camera reads within 0.26 ft RMSE of the USGS gauge (97% within 0.5 ft, $n = 30$), where a regression trained on normal flows misses by 6.3 ft. An event-gated agent (Claude Opus 4.6 on Amazon Bedrock) compares camera and gauge, gathers evidence with tools, and queues sensor flags and flood alerts for human approval. In replays with injected faults it catches all four gauge faults, two of them faster than a rule baseline, and correctly blames the *camera* in two camera-fault cases where the rules blame the healthy gauge. The system runs live on AWS Lambda (Graviton), EventBridge, S3 and Bedrock, and unchanged on a laptop CPU.**

## I. Introduction

Stage, the water level at a gauge, drives flood forecasts, warnings and discharge records. Gauges fail in quiet ways: a sensor drifts with sediment, a pressure line freezes, telemetry drops out. Hydrographers find these faults later, during record review; forecasters and emergency managers may act on the bad value in real time.

The USGS operates cameras at many of its gauges and serves their images through the NIMS API [1]. Published camera methods measure stage from such images with deep segmentation and photogrammetry [2], [3] or learned regression [4], [5], but they need surveyed control points, purpose-installed cameras or training labels from the very gauge one would want to check. None uses the camera to *audit* the gauge.

**Contributions.**

1) A gauge-free reader for staff-plate cameras: the plate's own marks give a 1-D projective pixel-to-feet map, the printed foot numbers (read with `cv.dnn.TextRecognitionModel`) give the datum, and per-mark colour contrast gives the waterline. It reads a flood never seen in setup at 0.26 ft RMSE and returns an explicit status instead of a guess when it cannot see.
2) An event-gated agent that arbitrates between two instruments: code detects disagreement, silence, blindness and flood levels; only then does Claude gather evidence (earlier frames, gauge history, an upstream gauge) and act. A rule planner with the same tools is the baseline.
3) A safe operating design: agent memory, a safety net that always surfaces flood alerts, a rule fallback, and a human approval queue; deployed serverless on AWS Graviton and evaluated with injected gauge and camera faults on a real flood, including the agent's failures.

## II. Related Work

Eltner et al. [2] segment water with a CNN and intersect the waterline with a photogrammetric 3-D model; Blanch et al. [3] run the same family for years at dedicated stations, which requires surveyed ground control and installed cameras. Vandaele et al. [4] transfer-learn water segmentation on public river cameras but stop at segmentation. Xu and Wang [5] assess deep networks that regress gauge height from webcam images; such models learn from the gauge record and are bounded by its range. StageSight reads an absolute stage from the plate itself, so it needs neither survey nor training labels, and it is designed to disagree with the gauge when the gauge is wrong.

## III. Data

**Camera.** The NIMS "Hancock Staff" camera, 1080×1920 frames, hourly, 21 Jul to 19 Oct 2026 (1,869 frames). The camera started on 14 Apr 2026; the 29 Jul event is its only flood. **Gauge.** USGS 01421500 gage height (parameter 00065, 15-min), used *only* for evaluation and by the agent as the instrument under test. **Upstream.** USGS 01417500 at Harvard, NY (agent tool; 90-day cross-correlation lag ≈3 h). **Flood levels.** NWS NWPS, HKCN6: action 14 ft, minor flood 18 ft. All data are public.

## IV. Reading the Plate with OpenCV 5

### A. Setup, once per camera

**Plate and marks.** Red pixels are found by an HSV threshold and a $3 \times 3$ opening; `cv.fitLine` with a Huber loss gives the plate axis $\mathbf{d}$ through origin $\mathbf{o}$. Each connected component is a mark; the long foot marks are kept (area $\geq 0.6\times$ the median of the eight largest) and projected onto the axis, $t_i = (\mathbf{c}_i - \mathbf{o})^\top \mathbf{d}$. Gaps of about twice the local spacing are indexed as a missing mark.

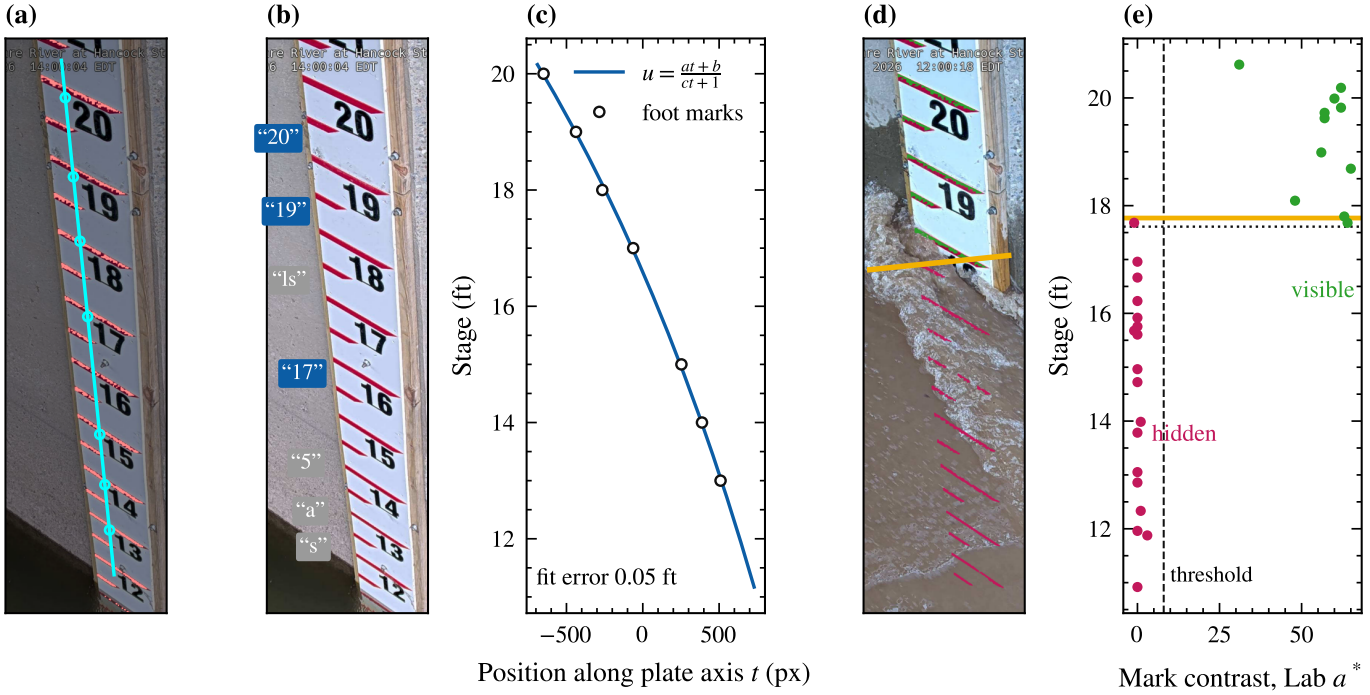



Fig. 1. The reader on real frames. Setup on a dry frame, no gauge data: (a) HSV red mask and robust plate axis (`cv.fitLine`, Huber) through the foot-mark centroids (`cv.connectedComponentsWithStats`); (b) CRNN text recognition of the printed numbers; blue reads vote for the datum, grey misreads cannot; (c) 1-D projective map from the foot marks. Every frame, here at the flood peak: (d) marks still visible (green) and hidden under water (magenta), waterline in yellow; (e) Lab $a^*$ contrast of each mark against its surround, on the stage axis: visible marks sit far above the threshold, submerged marks at zero. Reading 17.77 ft; gauge 17.61 ft.

**Scale.** A pinhole camera maps points on a straight plate to the image line by a 1-D homography, so plate units $u$ (ft below the top foot mark) follow

$$u(t) = \frac{a\,t + b}{c\,t + 1}, \qquad \begin{bmatrix} t_i & 1 & -t_i u_i \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = u_i, \tag{1}$$

solved by linear least squares over the foot marks (fit error 0.05 ft, Fig. 1c). Three parameters, fixed by geometry, extrapolate along the plate in a way a free polynomial does not.

**Datum.** The plate is rotated upright (`cv.warpAffine`); red is whitened and the black digits under each foot mark are cropped. The OpenCV Zoo CRNN [6], [7] runs through `cv.dnn.TextRecognitionModel` (CTC greedy decoding). Each digit read $k$ at mark $i$ votes for datum $D = k + u_i$. A dropped leading "1" is the usual misread, so only the longest reads may vote. At Hancock the vote gives $D = 20$ ft (6 of 7 votes). Stage is then $s = D - u$.

### B. Every frame

**Quality gate.** Frames that are blurred (variance of the Laplacian < 40), dark or over-exposed are `rejected`. **Registration.** The camera moves a few pixels between days. `cv.phaseCorrelate` on the always-dry top of the plate (Hann-windowed) gives the shift $(\Delta x, \Delta y)$; above 80 px the status is `camera_moved`. **Mark visibility.** For each of the $n = 29$ marks, the median Lab $a^*$ inside the mark is compared with a ring around it (`cv.dilate`, $15 \times 15$ minus $5 \times 5$):

$$v_j = \mathbf{1}\left[\mathrm{med}_{\mathrm{mark}\ j}\, a^* - \mathrm{med}_{\mathrm{ring}\ j}\, a^* \geq 8\right]. \tag{2}$$

The test is relative, so it holds in sun, shade and overcast; water and foam over a mark remove its contrast (Fig. 1e). **Waterline.** With marks ordered top to bottom, the split

$$k^\star = \arg\min_k \ \textstyle\sum_{j<k}(1 - v_j) + \sum_{j\geq k} v_j \tag{3}$$

tolerates a few wrong marks (glare, a leaf). The waterline $t^\star$ is the midpoint of the gap between mark $k^\star{-}1$ and mark $k^\star$, and $s = D - u(t^\star)$. **Status.** $k^\star \geq n - 1$ (only the lowest mark hidden, ambiguous at low water) gives `below_plate`; $k^\star = 0$ gives `unreadable` (night infrared, fog, spray). The reader never extrapolates below the plate.

## V. The Agent

OpenCV runs on every frame; Claude is called only when code raises an event (Table I). The planner receives the event, the open issues and its own recent actions (memory), and runs a manual tool loop with adaptive thinking through the Anthropic Bedrock client. Evidence tools: `reread_earlier_frames` (reruns the reader, including mark counts), `get_site_gauge`, `get_upstream_gauge`. Final actions: `flag_sensor` and `draft_alert` (both enter the approval queue),

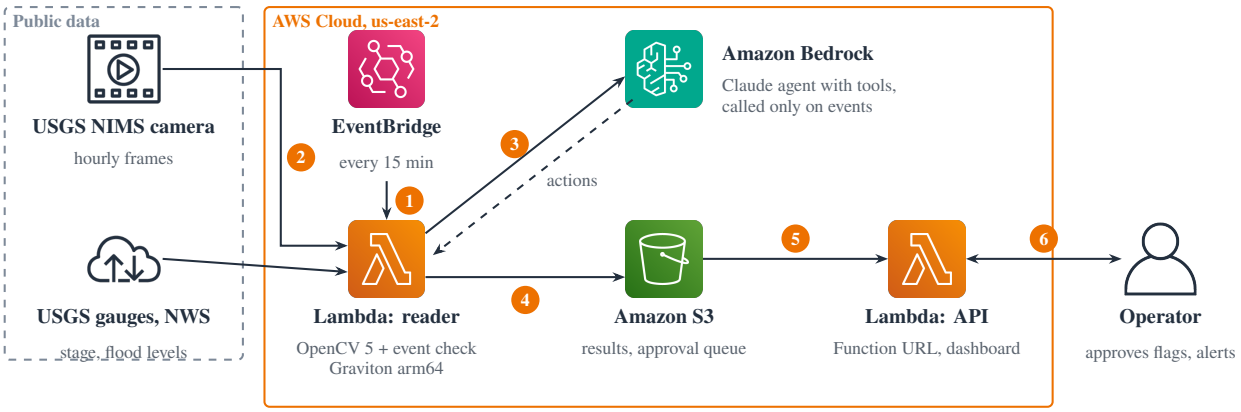



Fig. 2. Deployment. ① EventBridge starts the reader every 15 min; ② it fetches the newest frame and gauge values; ③ on an event it calls Claude on Bedrock; ④ results and the approval queue go to a private S3 bucket; ⑤–⑥ a second Lambda serves the dashboard and takes approvals.

TABLE I
EVENTS THAT CALL THE AGENT

| Event | Trigger (code, every frame) |
|---|---|
| `disagree` | camera and gauge differ > 0.5 ft for 2 frames |
| `gauge_silent` | camera reads, gauge has no recent value |
| `camera_blind` | camera unreadable 3 frames, gauge ≥ action |
| `level_rise` | camera or gauge reaches an NWS level |

TABLE II
STAGE ERROR ON THE 29 JULY 2026 FLOOD ($n$ = 30)

| Method | Gauge data | RMSE | Bias | ≤0.5 ft |
|---|---|---|---|---|
| **StageSight** (ours) | none | **0.26** | +0.07 | **97%** |
| Plate + gauge-fitted offset | training | 0.37 | −0.27 | 83% |
| Linear pixel fit, normal flows | training | 6.31 | −4.70 | 23% |

Errors in ft. StageSight MAE 0.21 ft, $R^2$ = 0.99.

`publish_provisional_stage`, `no_action`. Two operator facts are in the prompt: readings move in 0.5 ft steps, and upstream changes arrive about 3 h later.

**Safety design.** (i) *Safety net*: a `level_rise` always yields a flood alert draft for the operator; if the agent doubted it, its doubt is written into the draft. (ii) *Fallback*: on throttling, API errors or a refusal the rule planner decides, and the log says so. (iii) *Human in the loop*: no flag or alert leaves the system without approval.

## VI. System on AWS

Both functions run on arm64 (Graviton) from one 78 MB zip that bundles the Linux arm64 wheels of `opencv-python-headless` 5.0.0.93; it is built without Docker and deployed by an idempotent script that creates the bucket, an IAM role, both functions, the schedule and a Function URL (Fig. 2). From CloudWatch, a scheduled tick (download, read, agent check, write) takes a median of 6.3 s ($n$ = 23, peak memory 295 MB) and an API request 47 ms ($n$ = 1,227). At 96 ticks a day this is about 27,000 GB-s per month at the configured 1.5 GB. Claude is called only on events: none on a quiet day, two or three during the replayed flood, about seven tool steps each.

## VII. Experiments

### A. Accuracy on a real flood

Test frames are daylight frames in 21 Jul–1 Aug with the water on the plate ($n$ = 30). For the baselines, 15 normal-flow frames (up to 11.97 ft) serve as training data; 27 of the 30 test frames lie above that range.

Table II and Fig. 3: StageSight is the only method without gauge data and the most accurate. Marks are 0.5 ft apart, so a single reading is quantised to about ±0.25 ft; 0.26 ft RMSE is close to that floor. Fitting the offset to the gauge on normal flows is *worse* (0.37 ft): the lowest mark is often wet at low water, which biases the fit. The learned baseline fails because it never saw a flood. Two flood frames are regression tests in the repository (17.77 vs 17.61 ft; 12.63 vs 12.47 ft).

**Ablations.** A sub-mark edge search inside the bracket gave 0.275 ft against 0.257 ft for the midpoint and was not adopted. Using the bracket between two marks as an uncertainty interval covered the gauge in only 23–50% of frames, so we do not claim intervals.

### B. Coverage

Over 90 days the reader gave a number for 45 of 1,869 frames (2.4%, Fig. 4). The water was below the plate for 64% of frames and 33% were night infrared or fog. The plate is a flood instrument: it reads when the river is high, which is when a gauge fault matters.

### C. Agent: injected faults on the real flood

The replay feeds the 29 July flood frame by frame and injects four gauge faults and two camera faults (Fig. 6); the camera faults are synthetic overlays on real frames. The rule planner uses the same tools with fixed thresholds.

Both planners catch every gauge fault (Table III); Claude flags the frozen gauge in half the time (Fig. 5). The difference is in the camera faults: fixed thresholds see a persistent disagreement and blame the gauge, then publish the bad camera

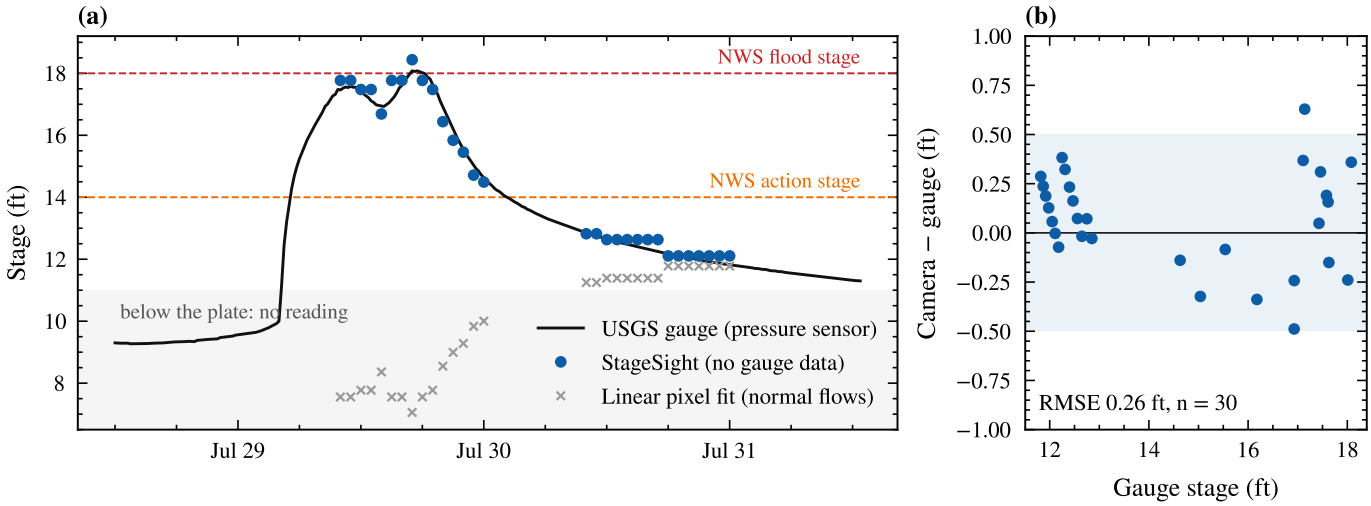



Fig. 3. (a) The camera follows both flood peaks with no gauge data; a linear fit trained on normal flows cannot reach flood stage. Below the plate (shaded) the reader reports `below_plate`. (b) Error against stage; the band is ±0.5 ft, the plate's half-foot mark spacing.

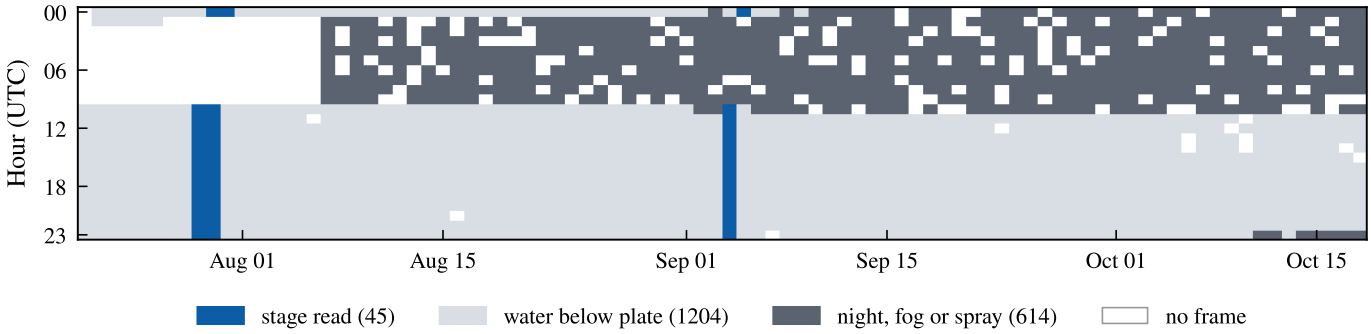



Fig. 4. Reader status for every hourly frame over 90 days (columns: days; rows: hour UTC). The camera reads during the two high-water periods; otherwise the water is below the plate (light) or it is night (dark).

TABLE III
RULES VS CLAUDE ON INJECTED FAULTS (SINGLE RUNS)

| Fault | Rule baseline | Claude Opus 4.6 |
|---|---|---|
| none (flood only) | 0 false flags, 2 alerts | 0 false flags, 2 alerts |
| gauge offset +1 ft | flagged, 1 h | flagged, 1 h |
| gauge silent 8 h | camera published, 1 h | camera published, 1 h |
| gauge frozen 10 h | flagged, 4 h | **flagged, 2 h** |
| gauge drift to +1.5 ft | flagged, 8 h | flagged, 7 h |
| debris over plate | blamed healthy gauge | **blamed camera** |
| spray on lens | blamed healthy gauge | **blamed camera** |

stage. Claude checks the evidence and blames the camera (Fig. 7).

In 90 days of normal operation (1,863 replayed ticks, rule planner) there were no false sensor flags and two alerts, both at the real flood.

### D. Development history of the agent

We report all three Claude runs. *Run 1* missed the drift (it blamed the camera); the two operator facts above were added. *Run 2* caught every fault but, in the frozen-gauge case, forgot its own gauge flag and drafted *no flood alert during a real flood*. The fix was in the system, not the prompt: memory of open issues and the safety net. *Run 3* (Table III): the safety net fired once, again in the frozen-gauge case: Claude reasoned from the 3 h lag that the river should be falling, but this flood peaked about 7 h after the upstream peak. We did not re-tune the lag to this flood. The evaluation has clear limits: one flood, one run per scenario, and prompt facts added after seeing run 1.

## VIII. LIMITATIONS AND FAILURE CASES

**Night**: infrared frames show the plate but not the red marks (`unreadable`). **Low water**: below about 11 ft the plate is dry and the camera cannot check the gauge. **Plate designs**: other NIMS plates (Owasco, Moravia, Chenango, NY) use black marks or 0.1 ft text, or stand on dry bank; they are not supported yet. **No plate**: a self-calibrated research mode learns a per-pixel flood-stage map from the archive (0.29 ft MAE in range on the Hancock wall) but does not extrapolate to floods; on far-bank bridge cameras (Hermann, MO; De Soto, KS; 2–3 px per ft) it reaches only 3.4 and 2.3 ft RMSE in range. **Model**: Claude Opus 4.6 is the most capable Claude

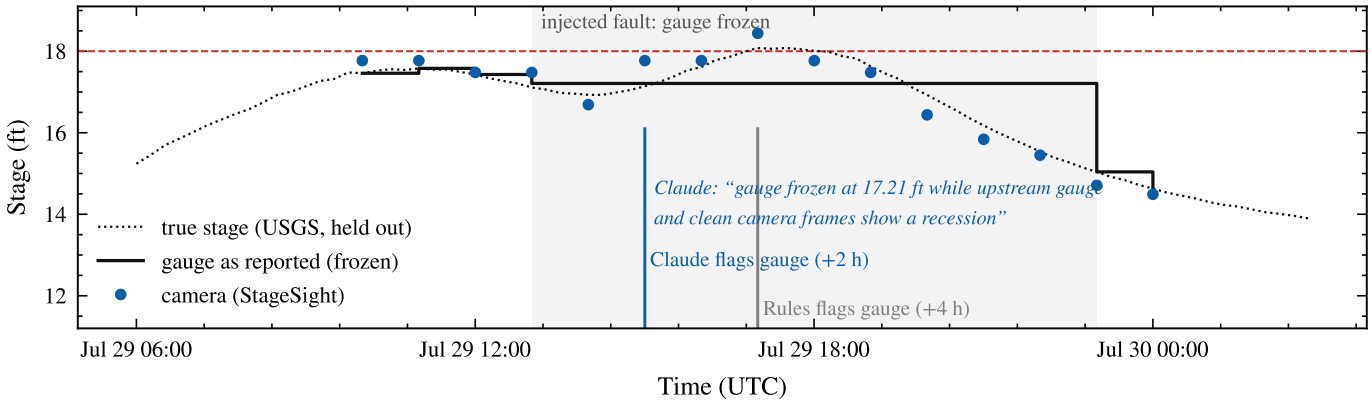



Fig. 5. Frozen-gauge replay. The reported gauge stays at 17.21 ft while the river rises to its second peak and falls. Claude flags the gauge 2 h after the fault, citing the upstream gauge and the camera; the rule baseline needs 4 h of consistent disagreement.

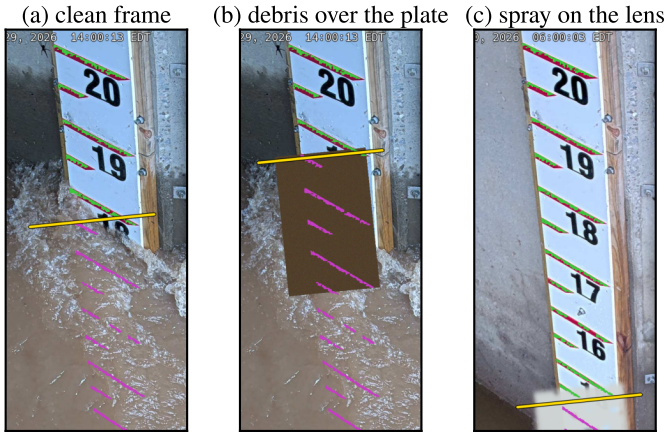



Fig. 6. Simulated camera faults on real frames, with the reader's output. (b) Debris hides the marks from 16.6 to 18.8 ft, (c) spray removes their colour; the reader reads too high and the gauge is right.

**Claude, debris case, 19:00 UTC (verbatim excerpt)**

**Tools:** `reread_earlier_frames(n=4)`, `get_site_gauge(12 h)`, `get_upstream_gauge(12 h)`.
**flag_sensor(camera):** "Camera reads 18.74 ft vs gauge 17.63 ft (+1.11 ft). Visible marks fell from 11 to 7 even as the gauge recedes smoothly from its 18.06 ft peak. Upstream gauge confirms falling trend with appropriate 3-hour lag."

Fig. 7. An agent decision in the debris case: the evidence Claude gathered and the flag it queued for approval.

model enabled on the new AWS account; the model is one setting.

## IX. Responsible Operation

StageSight is a decision aid, not an official warning system. Nothing is sent without a person's approval, and camera stages are labelled provisional. It uses public data only and no personal data. The bucket is private; the Lambda role can write only that bucket and invoke Bedrock models. The public Function URL serves read-only data; the approve endpoint is open in this demonstration and would sit behind authentication in operation. Every agent decision, tool call and reason is logged and shown on the dashboard.

## X. Conclusion

A camera that already watches a staff plate can measure stage by geometry and OCR alone, through a flood no learned model saw, and that independent reading lets an agent tell which instrument is wrong. Next: read night infrared frames from the printed numbers, support more plate designs across the NIMS network, and repeat the agent runs to report catch rates.

**Reproducibility.** Code, results and figures are at https://github.com/Waqar-msx9t/stagesight (Python 3.12, OpenCV 5.0.0.93, NumPy 2.5.4, Anthropic SDK 1.15.0). The plate tests run offline; three scripts (*plate_test*, *replay_agent*, *make_figures*) reproduce every number and figure.

## References

[1] U.S. Geological Survey, "National Imagery Management System (NIMS) API," https://api.waterdata.usgs.gov/nims/v0, accessed Oct. 2026.
[2] A. Eltner, P. O. Bressan, T. Akiyama, W. N. Gonçalves, and J. Marcato Junior, "Using deep learning for automatic water stage measurements," *Water Resources Research*, 2021, doi:10.1029/2020WR027608.
[3] X. Blanch, J. Grundmann, R. Hedel, and A. Eltner, "AI image-based method for a robust automatic real-time water level monitoring: a long-term application case," *Hydrology and Earth System Sciences*, vol. 30, p. 797, 2026, doi:10.5194/hess-30-797-2026.
[4] R. Vandaele, S. L. Dance, and V. Ojha, "Deep learning for automated river-level monitoring through river-camera images: an approach based on water segmentation and transfer learning," *Hydrology and Earth System Sciences*, vol. 25, p. 4435, 2021, doi:10.5194/hess-25-4435-2021.
[5] R. Xu and B. Wang, "Assessing deep learning techniques for remote gauging and water quality monitoring using webcam images," *Hydrology*, vol. 12, no. 4, p. 65, 2025, doi:10.3390/hydrology12040065.
[6] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," arXiv:1507.05717, 2015.
[7] OpenCV, "OpenCV Zoo: text recognition CRNN (text_recognition_CRNN_EN_2021sep)," https://github.com/opencv/opencv_zoo.